國立臺南大學資訊工程學系

資工三「演算法」課程
第 2 次作業

題目: Travel Map

班級 ： 資工三

姓名 ： 呂建篁

學號 ： S10859045

老師：陳宗禧

中華民國 110 年 11 月 14 日

目錄

(一) 簡介及問題描述……..……..……3

(二) 理論分析……..……4

(三) 演算法則……..……5

(四) 程式設計環境架構……..……6

(五) 程式……..……7~9

(六) 執行結果、討論與心得……..……10

參考文獻資料……..……...……11~12

(一) 簡介及問題描述

1. 簡介

給定一地圖，內有 n 個興趣點(POI, Point of Interest)，例：臺南古蹟、臺南小吃等。假設目 前有一個人或一個旅行團，對於臺南市的美食或古蹟有高度興趣，並且要規劃行程，但對於 該行程需要有資訊系統輔助，給定行前的建議，底下問題為規劃者需要知道的答案

2. 問題

Travel Map 實作與分析

(1) 哪兩個 POIs 靠最近? 距離多少? (brute-force algorithm)

(2) 這些 POIs 的範圍有多大 (Convex-Hull，它的面積以及最遠的距離)? (brute-force algorithm)

(3) (a) 假如有一個遊客(目前在某一個 POI a)要到所有 n 個 POIs，再回到原來的 POI a，則最短行程距離是多少(最佳解)? 時間複雜度? (TSP problem: exhaustive search + brute-force algorithm)

(b) 假如我們設計一個新演算法：(Convex-Hull-TSP Algorithm)

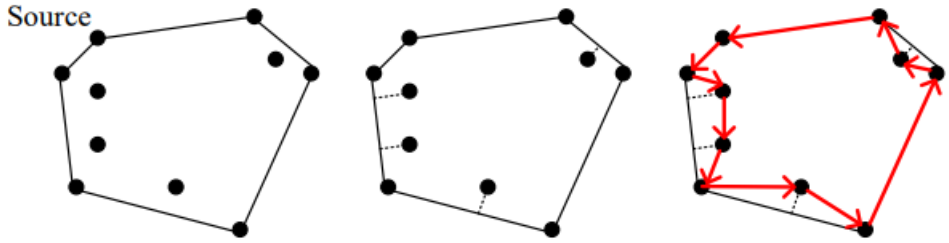
i. 求出所有點的 Convex-Hull

ii. 除 Convex-Hull 上的點外，其餘 POIs 找出離 Convex-Hull 邊最近的點投影

iii. 按照投影點依序由 Extreme Point 點開始旅遊，再繞回起始點

請問該演算法的時間複雜度(以 big O 表示)以及(行程)花多少距離?

Example:



(c) 比較 3(a)和 3(b)兩者的距離d_a和d_b、兩者比例$\frac{d_b}{d_a}$，以及比較兩者執行時間?

(4) 假設有 n 個遊客，因時間有限，想要最短總距離的方式分別到 n 個不同 POIs 旅遊，要如何配對，以及求出最少的總距離和? (Hungarian method)

(二) 理論分析

1. brute-force algorithm 理論

一種簡單且直接地解決問題的方法，常直接基於問題的描述和所涉及的概念定義。

2. exhaustive search 理論

一個關於搜尋特定元素的暴力破解法，先排列組合出所有可能性，再從中找出最佳解答。

3. Convex-Hull-Algorithm 理論

最小的凸多邊形集合，包含給定的點

Step1　先找出 extreme point 後連線$\overrightarrow{P1Pn}$

Step2　找到距離$\overrightarrow{P1Pn}$最遠的點($\overrightarrow{P1Pmax}$ 、 $\overrightarrow{PmaxPn}$)，並遞迴

Step3　將被包圍起來的點刪除，剩下的點即為凸多邊形的點。

4. Hungarian method 理論

給定一個$n \times n$的矩陣來找最佳的指派

Step1　找到每個列的最小值，並將每列裡的元素減去最小值

Step2　再找到每個欄的最小值，並將每欄的元素減去最小值

Step3　檢查各列，破上第一個零做記號，而同列或同欄其他零則畫θ

(由零較少的先做，可不依順序)

Step4　檢查可否完成僅含零的完全指派:若不能則用最少的刪除線來包含所有的零至少一次。

Step5　找出未被劃線的元素中最小值 k，將未被劃線的元素各列所有元素減去 k，若造成負值將負值該欄加上 k，形成新的矩陣後，回到 Step3。

(三) 演算法則

1. **Brute-force algorithm**

   time complexity: O(mn)

   space complexity: O(n)

2. **Exhaustive search Algorithm**

   time complexity: O(n)

3. **Convex-Hull-Algorithm (Quickhull)**

   time complexity: O(n^2)

   space complexity: O(n)

4. **Hungarian method Algorithm**

   time complexity: O(n^3)

(四) 程式設計環境架構

1. 程式語言

C++ in MS Windows

2. 程式開發工具

Visual C++ 2019 (第四題)

Dev C++ Version 2(第一題 & 第二題)

3. 電腦硬體

CPU: Intel Core i7 8550U

RAM: 8.00GB Single-Channel

4. 作業系統

Windows 10 家用版 64-bit

5. 顯示卡

Intel UHD Graphics 620

NVIDIA GeForce GTX 1050

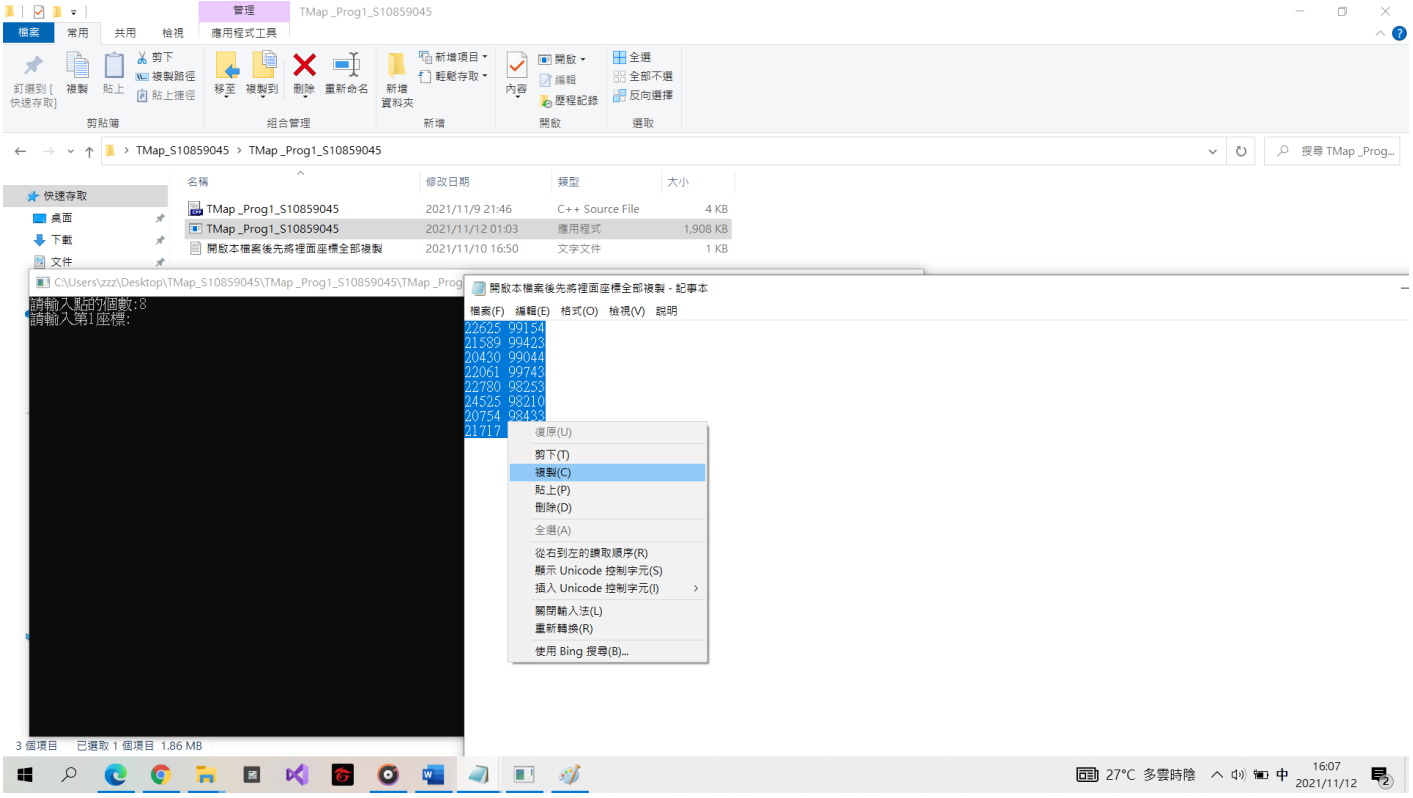
(五) 程式 (含 source code, input code, and output code)

1. 主程式

詳情請見 TMap_S10859045 資料夾裡的 cpp 檔

2. Input Code Format

[第一題] 將 txt 檔內的東西複製貼到 exe 執行檔即可完成座標輸入



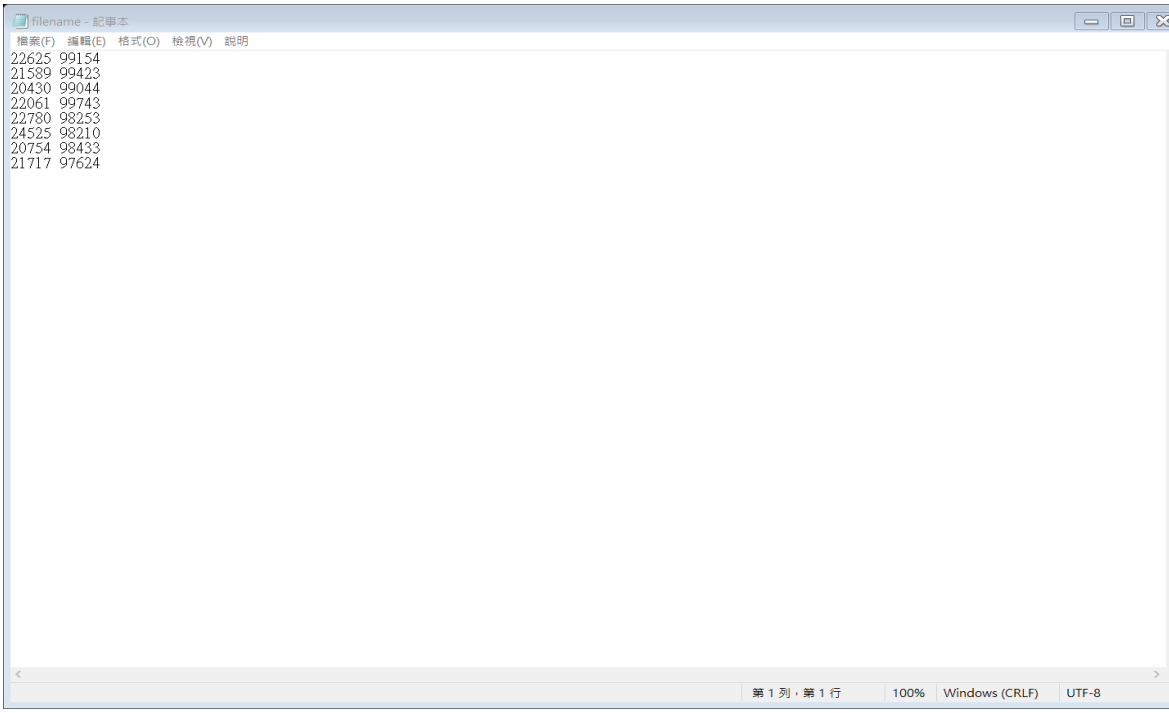
[第二題]

Filename.txt

會自動被讀入

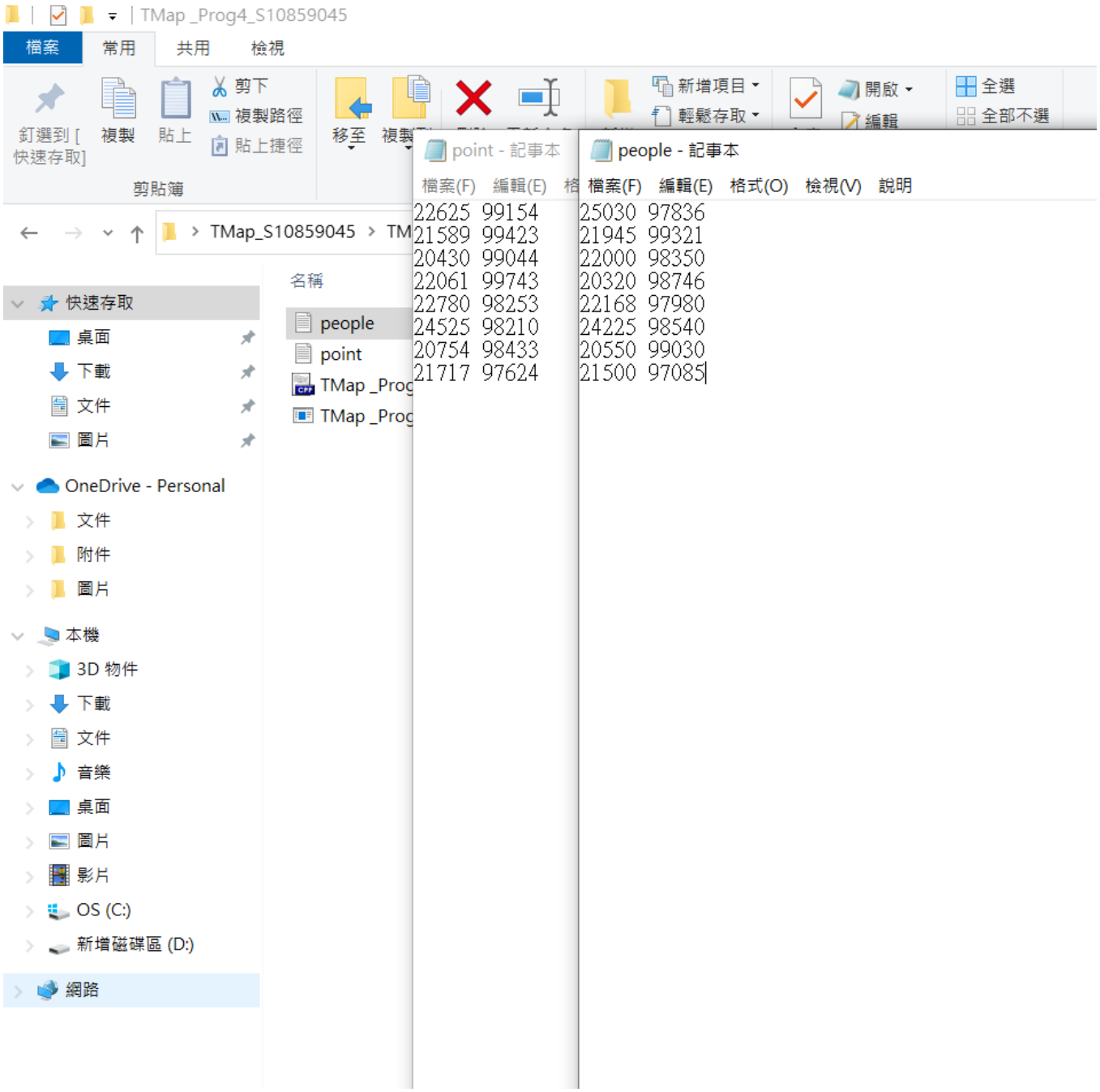
同個資料夾裡

的程式



[第四題]

Point 為景點檔案，People 為遊客座標位置 (Assume n=8)



3. Output Code Format

[第一題]

```
C:\Users\zzz\Desktop\TMap_S10859045\TMap _Prog1_S10859045\TMap _Prog1_S10859045.exe
請輸入點的個數:8
請輸入第1座標:22625 99154
請輸入第2座標:21589 99423
請輸入第3座標:20430 99044
請輸入第4座標:22061 99743
請輸入第5座標:22780 98253
請輸入第6座標:24525 98210
請輸入第7座標:20754 98433
請輸入第8座標:21717 97624
最小距離為: 570.249

最小距離的點為:(21589,99423),(22061,99743)
請按任意鍵繼續 . . .
```

[第二題]

```
C:\Users\zzz\Desktop\TMap_S10859045\TMap _Prog2_S10859045\TMap _Prog2_S10859045.exe
輸入的檔案資料偵測到8個點

在內部點有:(21589,99423) (22780,98253) (22625,99154)

包圍所形成Convex Hull的點:
 X座標  Y座標
(20754, 98433)
(21717, 97624)
(24525, 98210)
(22061, 99743)
(20430, 99044)
總面積為: 9.29048e+006
最長距離為:42958.342

請按任意鍵繼續 . . .
```

[第四題]

```
C:\Users\zzz\Desktop\TMap_S10859045\TMap _Prog4_S10859045\TMap _Prog4_S10859045.exe
從景點檔案讀取座標如下
景點1:(22625,99154)
景點2:(21589,99423)
景點3:(20430,99044)
景點4:(22061,99743)
景點5:(22780,98253)
景點6:(24525,98210)
景點7:(20754,98433)
景點8:(21717,97624)

第1位遊客的位置：(21500,97085)
第2位遊客的位置：(24225,98540)
第3位遊客的位置：(25030,97836)
第4位遊客的位置：(22000,98350)
第5位遊客的位置：(22168,97980)
第6位遊客的位置：(20550,99030)
第7位遊客的位置：(20320,98746)
第8位遊客的位置：(21945,99321)

最少的總距離之總和為： 14209.2

請按任意鍵繼續 . . .
```

(六) 執行結果、討論與心得

1. 執行結果

執行的結果符合題目所要求的，結果如(五)Output Code Format 所示

2. 討論

根據上面的理論演算分析可以得知，有部分題目使用暴力法，暴力法雖然容易實作且符合題目及定義所要求，但是其時間複雜度遠比其他演算法的方法技巧的時間複雜度還來的高。

3. 心得

藉本次演算法作業能更進一步了解 google map 是如何找尋日常生活中我們所遇到的問題並解決的，比如從臺南大學到臺南火車站之間的距離可以透過什麼方式來節省距離及時間，這時透過匈牙利或 Closest pair 的演算法實作分析，可以獲取最短距離來節省原先透過暴力法所花費的時間。

參考文獻資料

【Brute-force Algorithm】

https://ithelp.ithome.com.tw/articles/10236691

https://www-igm.univ-mlv.fr/~lecroq/string/node3.html

https://www.freecodecamp.org/news/brute-force-algorithms-explained/

https://www.quora.com/What-is-the-time-and-space-complexity-of-brute-force-algorithm

【Closest-Pair-Brute-force Algorithm】

https://www.796t.com/post/Ym54ZXk=.html

https://web.ntnu.edu.tw/~algo/AlgorithmDesign.html

https://blog.csdn.net/weixin_42887391/article/details/84638883

【Convex-Hull-Brute-force Algorithm】

https://web.ntnu.edu.tw/~algo/ConvexHull.html

https://iq.opengenus.org/divide-and-conquer-convex-hull/

https://blog.csdn.net/u013066730/article/details/106661030

https://www.geeksforgeeks.org/convex-hull-using-divide-and-conquer-algorithm/

https://www.tutorialspoint.com/convex-hull-using-divide-and-conquer-algorithm-in-cplusplus

https://www.linuxquestions.org/questions/programming-9/some-c-problems-convex-hull-area-etc-729712/

https://www.daniweb.com/programming/software-development/threads/365046/area-of-the-convex-hull-of-a-set-of-points

https://www.geeksforgeeks.org/maximum-distance-between-two-points-in-coordinate-plane-using-rotating-calipers-method/

【Hungarian Method】

https://codingnote.cc/zh-tw/p/173334/

https://codeforces.com/blog/entry/17511

https://brc2.com/the-algorithm-workshop/

https://github.com/mcximing/hungarian-algorithm-cpp

https://blog.csdn.net/u014754127/article/details/78086014

http://docs.ros.org/en/hydro/api/explorer/html/hungarian_8cpp_source.html

https://developers.google.com/optimization/reference/algorithms/hungarian

https://zh.wikipedia.org/zhtw/%E5%8C%88%E7%89%99%E5%88%A9%E7%AE%97%E6%B3%95

https://www.mathworks.com/matlabcentral/fileexchange/6543-functions-for-the-rectangular-assignment-problem

https://scicomp.stackexchange.com/questions/21241/looking-for-a-c-c-implementation-of-the-hungarian-method-for-real-valued-cost

【Exhaustive Search】

https://greedhead.net/which-problems-comes-under-exhaustive-search/

https://cstheory.stackexchange.com/questions/14404/complexity-class-associated-with-exhaustive-search